VIT VELLORE

School of Computer Science and Engineering

DATABASE SYSTEMS LAB

Review 2 – Implementation Report

Student Attendance Management System
using MongoDB

Student name	Harshit Verma
Registration number	24BKT0105
Programme	B.Tech CSE (Blockchain Technology)
Institution	VIT Vellore
Course	Database Systems Lab
Course code	BCSE302P
Review	Review 2 – Database Implementation and Core Functionalities
Database	StudentAttendanceDB (MongoDB)

Abstract

Manual attendance registers are slow to maintain, easy to get wrong and difficult to analyse. This project addresses that problem with a document database, **StudentAttendanceDB**, built on MongoDB. The database holds six collections – `users`, `students`, `faculty`, `courses`, `enrollments` and `attendance` – and is loaded with 105 sample documents, including 60 attendance records. Relationships between entities are expressed through identifier fields such as `student_id`, `faculty_id` and `course_id`.

This report documents the work completed for Review 2: creation of the database and collections, loading of sample data, CRUD operations (insert, find, update and delete), five aggregation pipelines (student-wise attendance percentage, course-wise statistics, students below 75% attendance, a daily summary and an absentee report with course details) and five supporting indexes. Each claim is tied to screenshots taken from the MongoDB Shell and MongoDB Compass, or to the project's JavaScript source file. Graph traversal and transactions were not implemented and are not claimed.

1. Introduction

Attendance is a basic academic record. It affects eligibility for examinations, drives interventions for students who are falling behind, and gives faculty an overview of class participation. In many departments it is still recorded on paper or in loosely organised spreadsheets. Such methods make it hard to answer even simple questions – for example, which students are below the required percentage, or how many students were absent on a given day – without repeated manual counting.

A database-backed system removes much of this effort. Records are stored once in a structured form, can be corrected in place, and can be summarised on demand using queries. MongoDB suits this use case well: each attendance entry, student or course is a self-contained document, and its aggregation framework can compute percentages, group records by course or date, and combine information from several collections in a single query [2].

2. Problem Statement

Attendance data for a class of students across several courses and dates grows quickly, and it is used in several different ways: a faculty member needs the record of one course, a student needs their own percentage, and an administrator needs lists of defaulters. The problem addressed here is to design and implement a database that (i) stores student, faculty, course, enrolment and attendance information in an organised way, (ii) supports creating, retrieving, updating and deleting records, (iii) can compute attendance statistics and identify students below a threshold without manual counting, and (iv) retrieves records efficiently as the volume of attendance data increases.

3. Objectives

The objectives of the project and their status at the time of Review 2 are listed in Table 1. An objective is marked "Implemented" only where the supplied source code and screenshots support it.

Table 1: Project objectives and their status at Review 2

Objective	Status	Where shown
Organise student, faculty, course, enrolment and attendance information in separate collections	Implemented	Section 5; Figures 1–3
Implement CRUD operations	Implemented	Section 7; Figures 4–5
Calculate student-wise attendance percentages	Implemented	Section 8.1; Figure 6
Identify students below a 75% attendance threshold	Implemented	Section 8.3; Figure 8

Objective	Status	Where shown
Generate course-wise and date-wise attendance summaries	Implemented	Sections 8.2 and 8.4; Figures 7 and 9
Produce an absentee report with student and course details	Implemented	Section 8.5; Figures 10–11
Improve retrieval efficiency using indexes	Indexes created; no performance measurement	Section 9; Figure 12
Graphical user interface, authentication and attendance-entry screens	Planned for Review 3 / future work	Section 11

4. Tools and Technologies

- **MongoDB** – the document database used to store all data. The screenshots show a local instance reached at `localhost:27017`.
- **MongoDB Compass** – the graphical client used to inspect the database, collection sizes, document counts and index counts (Figure 2).
- **MongoDB Shell (mongosh)** – the command-line interface, used inside Compass, through which the insert, find, update, delete, aggregation, index and count commands were run.
- **JavaScript** – the language of the shell commands. All commands are collected in one script, `mongodb_queries.js`, organised into eleven numbered sections.

No other framework, programming language or application server is part of the supplied evidence.

5. Database Design and Implementation

The database `StudentAttendanceDB` contains six collections, created by the first section of the script. Each document also carries the `_id` (ObjectId) field that MongoDB adds automatically, as visible in the screenshots. The fields in Table 2 are taken from the insertion code in `mongodb_queries.js`; for `students` and `attendance` they were additionally confirmed in the screenshots of query output.

Table 2: Collections, important fields and references

Collection	Purpose	Fields	References
`users`	Login identity and role (admin, faculty or student)	`user_id`, `username`, `role`, `active`	Referred to by `students.user_id`
`students`	Student profile	`student_id`, `name`, `email`, `branch`, `semester`, `user_id`	`user_id` → `users`
`faculty`	Teaching staff	`faculty_id`, `name`, `department`, `email`	Referred to by `courses.faculty_id`
`courses`	Courses offered	`course_id`, `course_name`, `faculty_id`, `credits`	`faculty_id` → `faculty`
`enrollments`	Which student is enrolled in which course	`enrollment_id`, `student_id`, `course_id`, `semester`, `status`	`student_id` → `students`; `course_id` → `courses`

Collection	Purpose	Fields	References
`attendance`	One record per student, course and date	`attendance_id`, `student_id`, `course_id`, `date`, `status`	`student_id` → `students`; `course_id` → `courses`

Relationships. Relationships are represented by storing the identifier of the related document, as shown in Figure 1. A student's account is linked through `user_id`; a course is linked to its faculty member through `faculty_id`; and the many-to-many relationship between students and courses is resolved by the `enrollments` collection, which holds one `student_id`/`course_id` pair per enrolment. Each attendance record names the student and the course it belongs to. MongoDB does not enforce these links as relational foreign-key constraints; they are logical references that are followed at query time with `$lookup` (Section 8). The only constraints enforced by the database itself are the unique indexes described in Section 9.

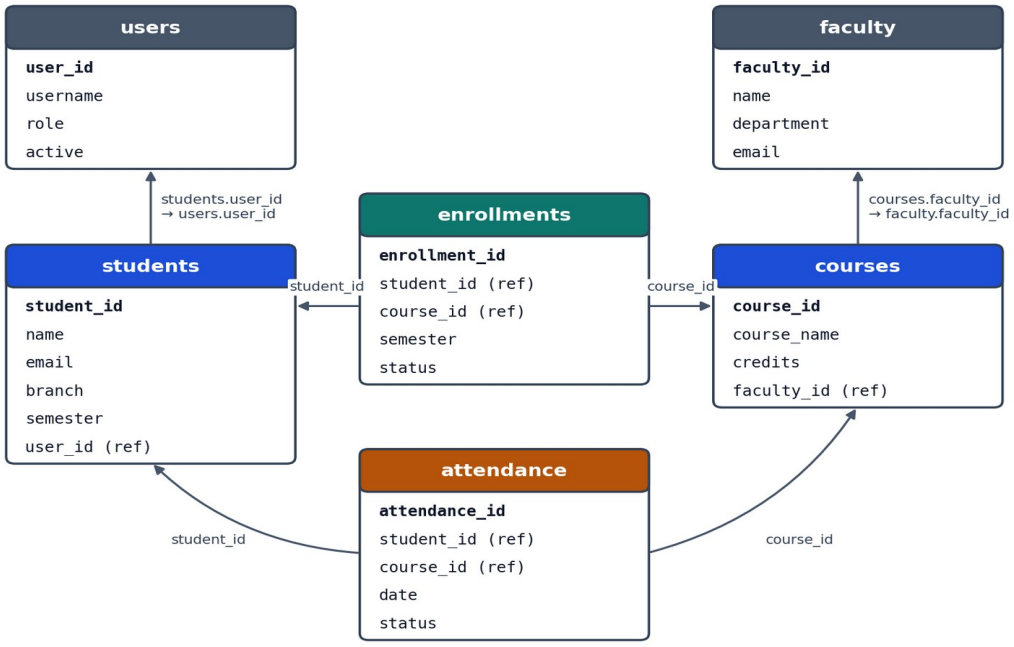


Figure 1: Logical relationships between the six collections of StudentAttendanceDB, drawn from the fields in the source script.

6. Sample Data and Database Validation

The sample data was loaded with `insertMany` from the script (Section 2 of `mongodb_queries.js`). It consists of the following:

- **Users (5):** `U001`–`U005` with the roles admin, faculty and student.
- **Students (10):** `S001`–`S010`, semester 5, with the branch alternating between BKT and CSE.
- **Faculty (5):** `F001`–`F005`, in the CSE and IT departments.
- **Courses (5):** `C001`–`C005` – Database Systems Lab, Operating Systems, Computer Networks, Compiler Design and Design and Analysis of Algorithms – each with 3 credits and one faculty member.
- **Enrollments (20):** every student is enrolled in `C001` and `C002`.

- **Attendance (60):** six class dates (1–6 September 2026) for each student, each marked Present or Absent. The attendance records cover `C001` and `C002` only, 30 records each (confirmed by Query 2 in Section 8).

The database overview in Compass (Figure 2) and the count commands run in the shell (Figure 3) both confirm the final document count of every collection. The numbers match the designed dataset exactly, and the total of 105 documents holds after the temporary test student created during the CRUD demonstration had been deleted.

Table 3: Document counts per collection (verified from Figures 2 and 3)

Collection	Designed	Verified (shell)	Compass overview
users	5	5	5
students	10	10	10
faculty	5	5	5
courses	5	5	5
enrollments	20	20	20
attendance	60	60	60
Total	**105**	**105**	**105**

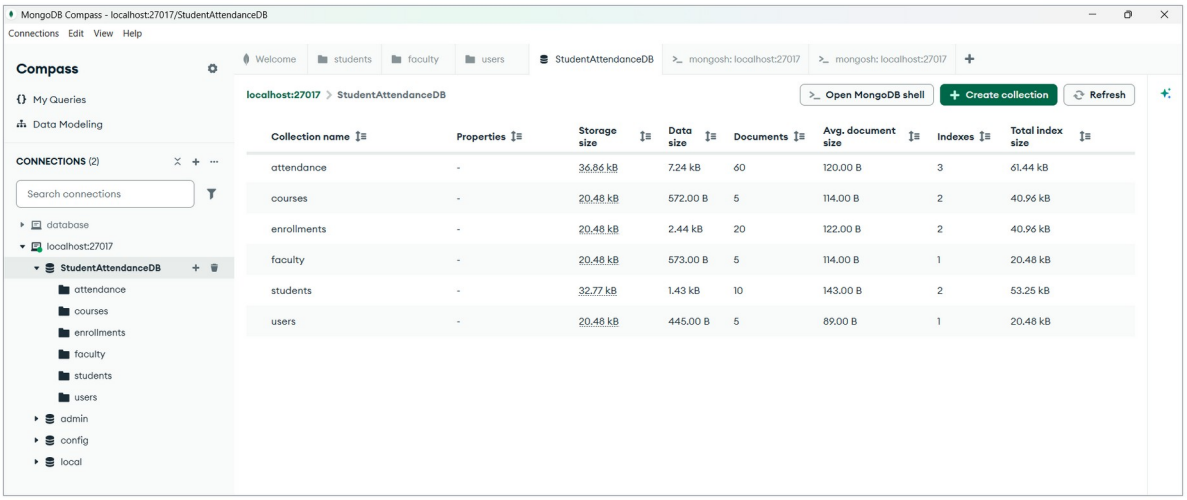


Figure 2: MongoDB Compass overview of StudentAttendanceDB showing the six collections, their document counts and index counts.



Figure 3: Record-count verification in the MongoDB Shell, ending with a total of 105 documents.

Adequacy of the dataset. The dataset exceeds the minimum requirement of 50 records. More importantly, it is varied enough to exercise every feature in this review: ten students in two branches, five courses linked to five faculty members, a many-to-many enrolment relationship, six dates, and a mixture of Present and Absent values. This produces non-trivial aggregation results – for example, students fall into two distinct percentage bands either side of the 75% threshold – and gives the unique and compound indexes realistic keys to work on.

7. CRUD Operations

The CRUD commands (Section 3 of the script) were run one by one in MongoDB Shell, using a temporary student, `S011`, as the test document so that the sample data stayed unchanged. Table 4 summarises them; the outputs are in Figures 4 and 5. The MongoDB manual describes these operations in [1].

Table 4: Summary of CRUD operations demonstrated

Operation	Command	Outcome shown in the screenshots
Create	`insertOne` on `students`	`acknowledged: true` with a new `insertedId`
Read	`find()`, `findOne({ student_id: "S001" })`, `find({ branch: "CSE" })`	Student documents returned; filter returned CSE students
Update	`updateOne` with `$set` on `students` and on `attendance`	`matchedCount: 1`, `modifiedCount: 1`; changed value confirmed by `findOne`
Delete	`deleteOne({ student_id: "S011" })`	`deletedCount: 1`

7.1 Create

A new student document was inserted into the `students` collection. This is the operation used when a student joins the course after the initial load.

```
db.students.insertOne({
  student_id: "S011", name: "Rahul Sharma", email: "rahul@example.com",
  branch: "CSE", semester: 5, user_id: "U003"
});
```

Figure 4(a) shows the command and its acknowledgement with the generated `insertedId`.

7.2 Read

Three read operations were executed: listing every student, retrieving one student by `student_id`, and filtering students by branch.

```
db.students.find();
db.students.findOne({ student_id: "S001" });
db.students.find({ branch: "CSE" });
```

Figure 4(a) also shows the beginning of the full student listing and Figure 4(b) shows the lookup of `S001` and the CSE filter. The filtered output contains the CSE sample students together with the temporary student `S011`, which existed at that point.

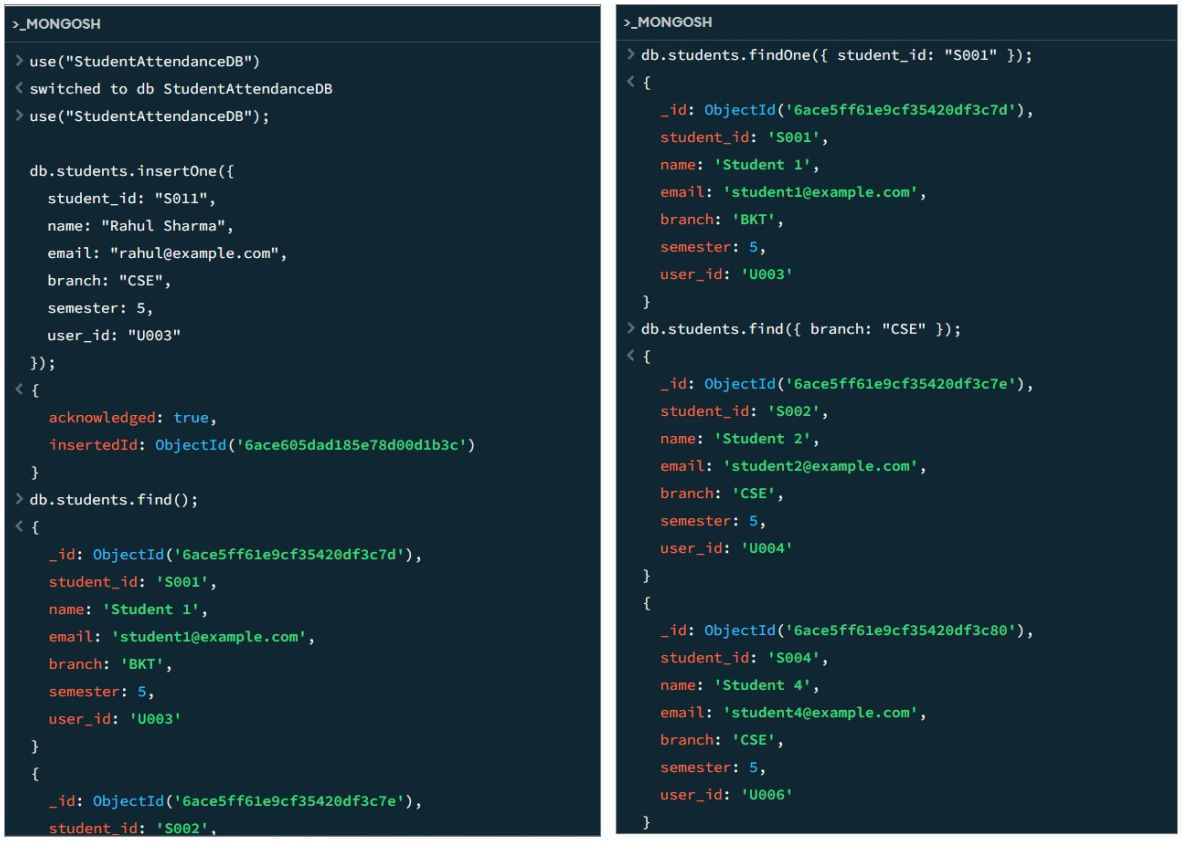


Figure 4: Create and Read operations: (a) insertion of student S011 and the start of the full listing; (b) retrieval of S001 by ID and filtering by branch CSE.

7.3 Update

Two updates were performed with the `$set` operator. First, the semester of the test student was changed from 5 to 6 and then checked with `findOne`. Second, attendance record `A001` was changed from Present to Absent, demonstrating how a correction to an attendance entry is made.

```
db.students.updateOne({ student_id: "S011" }, { $set: { semester: 6 } });
db.attendance.updateOne({ attendance_id: "A001" }, { $set: { status: "Absent" } });
```

Figure 5(a) shows `matchedCount: 1` and `modifiedCount: 1` for the student update, followed by a verification query returning `semester: 6`. Figure 5(b) shows the same acknowledgement for `A001` and the verified status `Absent`. This change to `A001` was left in place, so it is reflected in the aggregation results of Section 8 (student `S001` has 4 of 6 classes present).

7.4 Delete

The temporary test student was removed with `deleteOne`, returning the collection to its original ten documents.

```
db.students.deleteOne({ student_id: "S011" });
```

The command is visible at the bottom of Figure 5(a) and its result, `deletedCount: 1`, at the top of Figure 5(b). The later count of ten students in Figure 3 confirms the deletion.



Figure 5: Update and Delete operations: (a) update and verification of the student's semester, followed by the delete command; (b) delete acknowledgement and the attendance status update for A001.

8. Advanced NoSQL Features and Aggregation

The aggregation framework passes documents through a pipeline of stages, each transforming the output of the previous one [2]. The pipelines in this project use the following stages: `$match` filters documents; `$group` groups them and computes totals with `$sum`; `$cond` supplies a conditional value (1

if the status is “Present”, otherwise 0), which turns a count of present classes into a simple sum; `$addFields` adds computed fields such as percentages; `$lookup` combines each document with the matching documents of another collection; `$unwind` turns the matched array into a single embedded object; `$project` selects and renames the output fields; and `$sort` orders the results.

`$lookup` performs a join-like step inside the pipeline: for each input document it finds the documents in another collection whose `foreignField` equals the document’s `localField`, and adds them as an array [3]. It combines matching information at query time; it is not a relational foreign-key constraint, and it does not make MongoDB check that the referenced document exists.

Table 5 shows which stages each of the five queries uses.

Table 5: Pipeline stages used by each query

Query	$match	$group	$addFields	$lookup	$unwind	$project	$sort
1. Student-wise percentage	–	✓	✓	✓	✓	✓	✓
2. Course-wise statistics	–	✓	–	✓	✓	✓	✓
3. Below 75%	✓	✓	✓	✓	✓	✓	✓
4. Daily summary	–	✓	–	–	–	–	✓
5. Absentees with course	✓	–	–	✓ (2)	✓ (2)	✓	✓

Queries 1 to 4 also use `$sum` with `$cond` inside `$group` to count present and absent records. Every query reads from the `attendance` collection. The outputs shown below come from the run of the script that matches the source file (including its `$sort` stages).

8.1 Student-wise attendance percentage

Problem solved. It gives each student’s number of classes, number of classes attended and percentage of attendance, with names, so that students can be compared and ranked. **Stages.** `$group` by `student_id` counts all records and, using `$sum` with `$cond`, the present ones; `$addFields` computes the percentage rounded to two decimal places; `$lookup` and `$unwind` bring in the student’s name from `students`; `$project` shapes the output; `$sort` orders by percentage, highest first. **Practical use.** A per-student attendance report for faculty and students.

```
db.attendance.aggregate([
  { $group: { _id: "$student_id", totalClasses: { $sum: 1 },
      presentClasses: { $sum: { $cond: [{ $eq: ["$status", "Present"] }, 1,
0] } } } },
  { $addFields: { attendancePercentage: { $round: [
      { $multiply: [{ $divide: ["$presentClasses", "$totalClasses"] }, 100] },
2] } } },
  { $lookup: { from: "students", localField: "_id",
               foreignField: "student_id", as: "studentDetails" } },
  { $unwind: "$studentDetails" },
  { $project: { _id: 0, student_id: "$_id", name: "$studentDetails.name",
                totalClasses: 1, presentClasses: 1, attendancePercentage: 1 } },
  { $sort: { attendancePercentage: -1 } }
]);
```

Output. The query returned one result for each of the ten students, each with six classes (Figure 6 and the continuation of the output in the source screenshots). Four students – `S004`, `S005`, `S008` and `S009` – attended 5 of 6 classes (83.33%); six students – `S001`, `S002`, `S003`, `S006`, `S007` and `S010` – attended 4 of 6 classes (66.67%).



Figure 6: Student-wise attendance percentage: end of the pipeline and the first results, sorted by percentage in descending order.

8.2 Course-wise attendance statistics

Problem solved. It summarises attendance for each course: total records, present count and absent count, together with the course name. **Stages.** `$group` by `course_id` with `$sum`/`$cond`; `$lookup` and `$unwind` against `courses` for `course_name`; `$project`; `$sort` by `course_id`. **Practical use.** It lets a faculty member or coordinator compare attendance across courses.

```
db.attendance.aggregate([
  { $group: { _id: "$course_id", totalRecords: { $sum: 1 },
      presentCount: { $sum: { $cond: [{ $eq: ["$status", "Present"] }, 1, 0] } },
      absentCount:  { $sum: { $cond: [{ $eq: ["$status", "Absent"] }, 1, 0] } } } },
  { $lookup: { from: "courses", localField: "_id",
               foreignField: "course_id", as: "courseDetails" } },
  { $unwind: "$courseDetails" },
  { $project: { _id: 0, course_id: "$_id", course_name: "$courseDetails.course_name",
                totalRecords: 1, presentCount: 1, absentCount: 1 } },
  { $sort: { course_id: 1 } }
]);
```

Table 6: Output of the course-wise query (Figure 7)

Course	Course name	Records	Present	Absent
C001	Database Systems Lab	30	22	8
C002	Operating Systems	30	22	8

Only `C001` and `C002` appear because the sample attendance records exist for these two courses only. Together they account for all 60 attendance records (44 present, 16 absent).



Figure 7: Course-wise attendance statistics: the end of the pipeline and the two result documents.

8.3 Students below 75% attendance

Problem solved. It identifies students whose attendance has fallen below the 75% threshold, so that they can be warned or counselled. **Stages.** `$group` and `$addFields` compute each student's percentage; `$match` keeps those with `percentage < 75`; `$lookup` and `$unwind` add the name; `$project` rounds the percentage to two decimals; `$sort` lists the lowest first. **Practical use.** A defaulter list that can be produced at any time.

```
db.attendance.aggregate([
  { $group: { /* totalClasses and presentClasses, as in Query 1 */ } },
  { $addFields: { percentage: { $multiply: [
      { $divide: ["$presentClasses", "$totalClasses"] }, 100] } } },
  { $match: { percentage: { $lt: 75 } } },
  { $lookup: { from: "students", localField: "_id",
               foreignField: "student_id", as: "student" } },
  { $unwind: "$student" },
  { $project: { _id: 0, student_id: "$_id", name: "$student.name",
                attendancePercentage: { $round: ["$percentage", 2] } } },
  { $sort: { attendancePercentage: 1 } }
]);
```

Output. Six students were returned, each with 66.67%: `S003`, `S001`, `S002`, `S006`, `S007` and `S010` (Figure 8). These are exactly the six students in the lower band of Query 1; the four students at 83.33% are above the threshold and are correctly excluded.

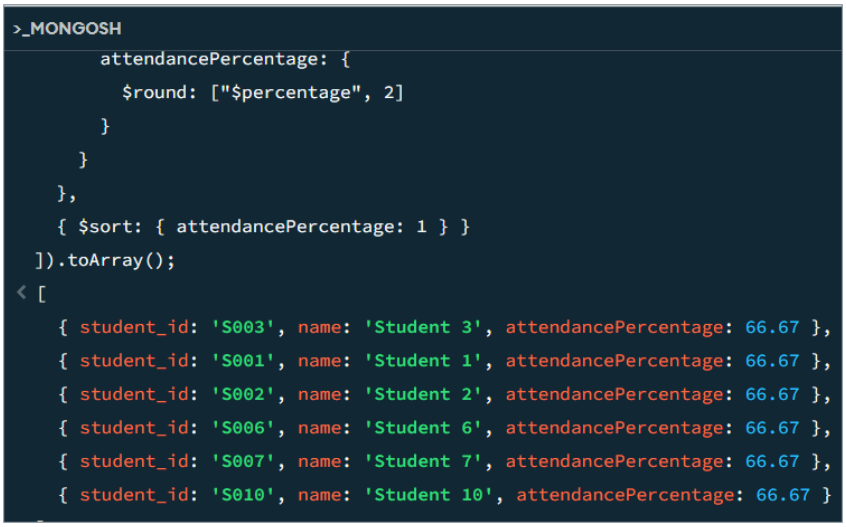


Figure 8: Students with attendance below 75%: end of the pipeline and the six matching students.

8.4 Daily attendance summary

Problem solved. It shows, for each class date, how many attendance records were Present and how many were Absent. **Stages.** `$group` by `date` using `$sum`/`$cond`, then `$sort` by date. No `$lookup` is needed because the date is stored in the attendance record itself. **Practical use.** A day-by-day view of attendance trends.

```
db.attendance.aggregate([
  { $group: { _id: "$date", totalRecords: { $sum: 1 },
      present: { $sum: { $cond: [{ $eq: ["$status", "Present"] }, 1, 0] } },
      absent:  { $sum: { $cond: [{ $eq: ["$status", "Absent"] }, 1, 0] } } } },
  { $sort: { _id: 1 } }
]);
```

Table 7: Output of the daily summary (Figure 9)

Date	Total records	Present	Absent
2026-09-01	10	7	3
2026-09-02	10	7	3
2026-09-03	10	7	3
2026-09-04	10	8	2
2026-09-05	10	8	2
2026-09-06	10	7	3
Total	**60**	**44**	**16**



Figure 9: Daily attendance summary: the end of the pipeline and the six daily results.

8.5 Absent students with course details

Problem solved. It produces a readable absentee report: for each Absent record, the student's name, the course name and the date. **Stages.** `$match` selects `status: "Absent"`; two `$lookup` stages (against `students` and `courses`) followed by two `$unwind` stages attach the related documents; `$project` keeps four fields; `$sort` orders by date. **Practical use.** A report that can be given to a class coordinator or used to notify students.

```
db.attendance.aggregate([
  { $match: { status: "Absent" } },
  { $lookup: { from: "students", localField: "student_id",
               foreignField: "student_id", as: "student" } },
  { $lookup: { from: "courses", localField: "course_id",
               foreignField: "course_id", as: "course" } },
  { $unwind: "$student" }, { $unwind: "$course" },
  { $project: { _id: 0, student_name: "$student.name",
                course_name: "$course.course_name", date: 1, status: 1 } },
  { $sort: { date: 1 } }
]);
```

Output. The pipeline returned 16 absent records, matching the 16 absences counted in Queries 2 and 4. Table 8 groups them by date; the full listing runs across Figures 10 and 11 and the following screenshot in the source document.

Table 8: Absent records returned by the absentee report

Date	Students absent	Course
2026-09-01	Student 1, Student 3, Student 7	Operating Systems
2026-09-02	Student 2, Student 6, Student 10	Database Systems Lab
2026-09-03	Student 1, Student 5, Student 9	Operating Systems
2026-09-04	Student 4, Student 8	Database Systems Lab
2026-09-05	Student 3, Student 7	Operating Systems
2026-09-06	Student 2, Student 6, Student 10	Database Systems Lab



Figure 10: Absent students with course details: the end of the pipeline and the first absent records (1–2 September).

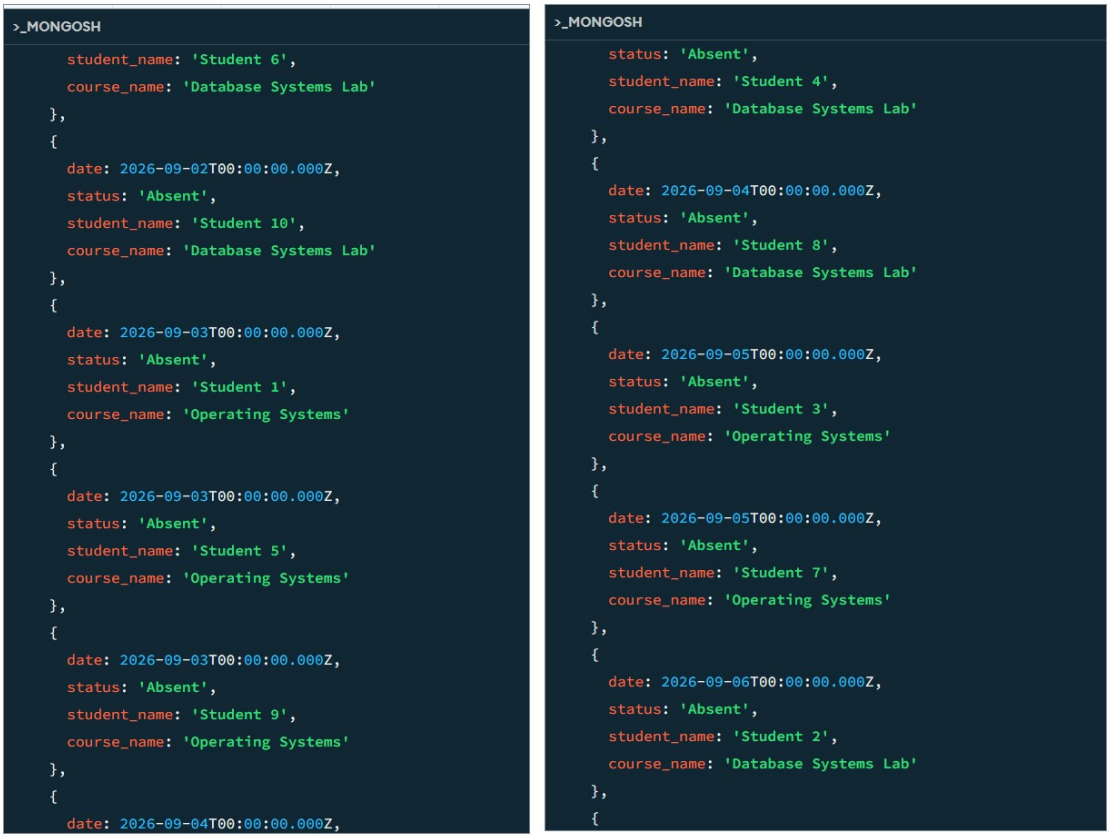


Figure 11: Absent students with course details: later records (2–6 September).

9. Indexing and Query Efficiency

An index is an additional data structure that lets MongoDB locate documents matching a query without scanning the whole collection [4]. Section 9 of the script defines five indexes; they are listed in Table 9 with the purpose of each. The definitions below were checked against the source file, and they match the commands shown in the screenshots.

```
db.students.createIndex({ student_id: 1 }, { unique: true });
db.courses.createIndex({ course_id: 1 }, { unique: true });
db.attendance.createIndex({ student_id: 1, course_id: 1, date: 1 });
db.attendance.createIndex({ status: 1 });
db.enrollments.createIndex({ student_id: 1, course_id: 1 }, { unique: true });
```

Table 9: Indexes defined in the project

Collection	Key (options)	Purpose	Evidence
`students`	`student_id` (unique)	Fast lookup of a student by ID; rejects duplicate IDs; supports the `$lookup` from `attendance` in Queries 1, 3 and 5	Command run; index count 2 in Compass (Figure 2)
`courses`	`course_id` (unique)	Fast lookup of a course by ID; rejects duplicate IDs; supports `$lookup` in Queries 2 and 5	Command run; index count 2 in Compass (Figure 2)
`attendance`	`student_id`, `course_id`, `date` (compound)	Supports queries on a student, on a student within a course, and on a student within a course on a date [5]	`getIndexes()` output (Figure 12)
`attendance`	`status`	Supports filtering by Present or Absent, as in the `$match` of Query 5	`getIndexes()` output (Figure 12)
`enrollments`	`student_id`, `course_id` (unique compound)	Prevents the same student from being enrolled twice in the same course; supports enrolment lookups	`getIndexes()` output (Figure 12)

Compass reports three indexes for `attendance` (the default `_id` index plus the two above), two each for `students`, `courses` and `enrollments`, and one each for `users` and `faculty` (Figure 2). Figure 12 displays the full `getIndexes()` output for `attendance` and `enrollments`, including `unique: true` on the enrolment index. The shell output for `students` and `courses` is not included in the screenshots, so their uniqueness is evidenced by the executed `createIndex` commands and the index counts rather than by a printed index definition.



Figure 12: Index creation and verification: (a) createIndex commands and the start of the getIndexes() output for attendance; (b) the remaining attendance index (status_1) and the unique compound index on enrollments.

Efficiency. Indexes help only queries that match their keys, and a compound index supports queries on the leading fields of its key. The sample data (105 documents) is too small for any difference to be noticeable, and no `explain()` plans or timings were recorded. Therefore no speed-up is claimed; the indexes are described as the supporting structures intended for larger attendance volumes.

10. Results and Discussion

Table 10 relates the results to the Review 2 rubric.

Table 10: Mapping of results to the Review 2 rubric (10 marks)

Criterion	Marks	Evidence
Database implementation	2	Six collections created and populated (Figures 2–3); relationships represented by identifier fields (Figure 1, Table 2); 105 verified documents, exceeding the 50-record minimum (Table 3)
CRUD operations	3	Insert, three read variants, two updates and a delete executed successfully (Figures 4–5, Table 4)
Advanced NoSQL features	5	Five aggregation pipelines with `$group`, `$lookup`, `$unwind` and other stages, each executed with results (Figures 6–11); five indexes defined, with output shown for attendance and enrollments (Figure 12)

The results are internally consistent. All of the following agree with each other: 60 attendance records; 44 present and 16 absent in both Query 2 and Query 4; 16 rows in the absentee report; six students below 75% in Query 3 corresponding to the six students at 66.67% in Query 1. The update of `A001` to Absent in Section 7.3 is reflected in the results (student `S001` at 4 of 6 classes). Together these show that the stored data, the CRUD operations and the aggregation logic behave as designed.

Two advanced features that the rubric mentions – graph traversal and transactions – were not implemented, and this report does not claim them. The advanced-feature evidence consists of the aggregation pipelines and the indexes.

11. Limitations and Future Work

Current limitations

- The project is limited to database implementation and query logic; there is no graphical interface or application layer.
- The `users` collection stores roles, but authentication and role-based access are not integrated into any application.
- References between collections are not enforced by the database. **Sample-data consistency:** in the executed data, students `S004`–`S010` carry `user_id` values `U006`–`U012` (visible for several students in Figure 4(b)), whereas the script defines users `U001`–`U005` only and generates student `user_id` values cycling through `U003`–`U005`. This does not cause an error, but the sample data should be made consistent before final submission.
- Attendance exists for two of the five courses, and no schema validation or error handling beyond the unique indexes has been added.
- No performance measurements (for example `explain()` plans) have been recorded for the indexes.

Future work (planned, not yet implemented)

- An application or web interface for entering attendance, with input validation and error handling.
- Integration of authentication and role-based access using the `users` collection.
- Schema validation rules and checks that references (`student_id`, `course_id`, `user_id`) point to existing documents.
- Attendance data for all five courses, and performance testing of the indexes with `explain()` on a larger dataset.
- Evaluation of MongoDB multi-document transactions for operations that update several collections together.

12. Conclusion

For Review 2, the Student Attendance Management System has a working MongoDB database with six collections, a verified sample dataset of 105 documents, a complete set of CRUD operations, five aggregation pipelines that produce student-wise, course-wise, defaulter, daily and absentee reports, and five indexes aimed at the most common lookups. The evidence in this report covers each of the three rubric areas: database implementation, CRUD operations and advanced NoSQL features. The next step towards Review 3 is to move from shell scripts to a usable system: an interface for entering and viewing attendance, authentication and role-based access, stronger validation, and a more consistent and larger sample dataset.

References

[1] MongoDB Inc., “MongoDB CRUD Operations,” MongoDB Manual. https://www.mongodb.com/docs/manual/crud/

[2] MongoDB Inc., “Aggregation Pipeline Stages,” MongoDB Manual. https://www.mongodb.com/docs/manual/reference/operator/aggregation-pipeline/

[3] MongoDB Inc., “$lookup (aggregation),” MongoDB Manual. https://www.mongodb.com/docs/manual/reference/operator/aggregation/lookup/

[4] MongoDB Inc., “Unique Indexes,” MongoDB Manual. https://www.mongodb.com/docs/manual/core/index-unique/

[5] MongoDB Inc., “Compound Indexes,” MongoDB Manual (v5.0). https://www.mongodb.com/docs/v5.0/core/index-compound/

Appendix: Source Files and Repository Contents

Table 11: Files supporting this report

File	Description
`mongodb_queries.js`	MongoDB Shell script with eleven sections: (1) database and collection creation, (2) sample data insertion, (3) CRUD operations, (4–8) five aggregation queries, (9) index creation, (10) index verification, (11) record-count verification. The sample-data section is guarded so that deleting existing documents must be enabled deliberately.
`screenshots.docx`	Word document containing the 51 screenshots of the MongoDB Compass and MongoDB Shell sessions from which the figures in this report are taken.
`README.md`	Prepared by the student for the GitHub repository; not part of the material reviewed for this report.

Repository URL: not supplied with the report material – insert the GitHub link here before submission.